



OSTBAYERISCHE
TECHNISCHE HOCHSCHULE
REGENSBURG

ELEKTRO- UND
INFORMATIONSTECHNIK

PRAKTIKUM INFORMATIK 1

L. Osinski, T. Langer, M. Niemetz

Programmierwerkzeuge - Teil B

Oktober 2025, Version: 2.31

Fakultät Elektro- und Informationstechnik

1 Einleitung

Zur Bearbeitung der in diesem Praktikum auf Sie zukommenden Programmieraufgaben benötigen Sie eine große Vielfalt entsprechender Werkzeuge wie Präprozessor, Compiler, Linker und Debugger, die Sie ja in der vorangehenden Aufgabe bereits kurz kennengelernt haben.

Um den Umgang mit diesen Werkzeugen zu vereinfachen und eine möglichst konsistente Benutzerschnittstelle über verschiedene Compiler für verschiedene Zielplattformen hinweg zu bieten, existieren sogenannte „Integrated Development Environments“ (IDE). Diese nehmen Ihnen sehr viel Arbeit ab und verstecken die Vielfalt an Werkzeugen weitgehend unter einer komfortablen Oberfläche.

In dieser Aufgabe sollen Sie die Chance bekommen, Ihr zur Verfügung stehendes Werkzeug weiter kennen zu lernen mit dem Ziel, später auch effizient damit arbeiten zu können.

Wichtig: Bearbeiten Sie diese Aufgabe möglichst gründlich und machen Sie sich mit Ihrem Programmierwerkzeug intensiv vertraut. Damit sparen Sie im weiteren Verlauf des Semesters wertvolle Zeit.

Hinweis: Die hier verwendete IDE CodeBlocks ist nicht nur portabel, kostenlos und ohne Registrierung nutzbar, sondern auch sehr minimal gehalten. Das ist Absicht, um Sie nicht vom Wesentlichen, dem Lernen der Grundprinzipien, abzulenken.

1.1 Kennenlernen der IDE

1.1.1 Verzeichnis einrichten

Wie Sie bereits wissen, entsteht für jedes Programm während des Kompilierens und Linkens eine Reihe weiterer Dateien. Aus diesem Grund ist es sinnvoll, für jede Aufgabe bzw. jedes Programm ein eigenes Unterverzeichnis („Projektverzeichnis“) mit einem passenden Namen zu erstellen.

Hierfür können Sie das in der vorangehenden Aufgabe angelegte Verzeichnis als Wurzelverzeichnis verwenden, in welchem Sie dann die Verzeichnisse für die einzelnen Projekte erzeugen.

Wichtig: Achten Sie darauf, im gesamten Dateipfad (also im Dateinamen und allen Namen der Verzeichnisse, durch die man sich bewegen muss, um zur Datei zu gelangen) **weder Leerzeichen noch Sonderzeichen wie z.B. Umlaute,**

sondern ausschließlich Buchstaben, Ziffern und Unterstriche zu verwenden.

Beachten Sie dies nicht, wird der Debugger nicht oder nicht richtig funktionieren. Punkt.

1.1.2 Projekt einrichten

Um alle für eine Programmieraufgabe benötigten Dinge zusammenzuhalten, verwenden IDEs sogenannte Projekte. Diese sind wie eine Stückliste zu verstehen, die alle für das Projekt benötigten Komponenten und Konfigurationen zusammenfassen.

Hinweis: IDEs speichern diese Projektinformationen typischerweise in Projektdateien ab. Diese enthalten aber nicht den gesamten Programmcode, so wie Sie das z.B. von den Dateien der Textverarbeitungen her kennen, sondern nur die Projekteinstellungen und Stücklisten. Die Code-Dateien werden also nur referenziert. Der Grund hierfür ist die damit effizientere Verarbeitung und Verwaltung des Programmcodes.

Ein Projekt gibt insbesondere an

- In welchem Verzeichnis die Programme und Hilfsdateien liegen,
- Welcher Typ von Programm erstellt werden soll, z.B. Bibliothek, Console Application, etc.,
- Welche Dateien in die Erstellung des Programms einbezogen werden sollen
- Welche Einstellungen für die Übersetzung und den Linker-Lauf verwendet werden sollen

Wichtig: Öffnen Sie **niemals die Einzeldateien** Ihres Projekts direkt in der IDE, sondern **immer das Projekt** als Ganzes über die Projektdatei. Befolgen Sie dies nicht, wird Ihre IDE nicht oder nicht richtig funktionieren.

Projekt anlegen Der Erstellungsassistent für ein Projekt wird über den Menüpunkt *File/New/Project* gestartet. Als erstes erscheint der Dialog zur Auswahl des Projekttyps (vgl. Abb. 1.1.2). Für dieses Praktikum kommen die drei folgenden Möglichkeiten in Frage:

- **Console Application:** für Programme, die im Textmodus laufen, z.B. die ersten Übungsaufgaben
- **Empty Project:** für Programme, bei denen alle Programmdateien für die Übung bereits vorgegeben sind, z.B. WinA-dapt Projekte

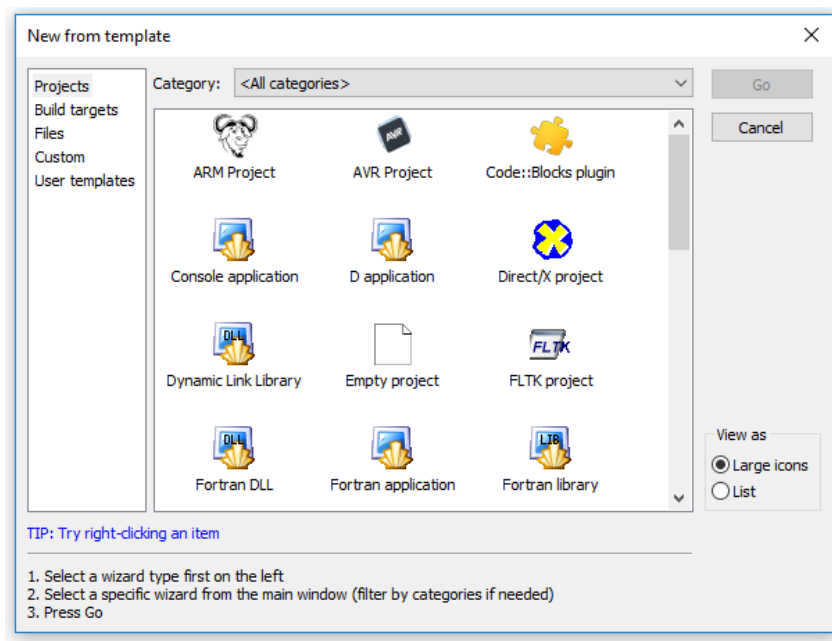


Abbildung 1: Dialog in CodeBlocks zur Auswahl des Projekttyps bei der Erzeugung eines neuen Projektes.

- **Win32 GUI Project** für Programme mit Grafikausgabe ohne WinAdapt

Name und Verzeichnis Nach dem Projekttyp-Dialog und der Wahl der Sprache: C oder C++ (vgl. Abb. 2) muss der Titel: `Project title` und der Speicherort des Projekts: `Folder to create project in` angegeben werden (vgl. Abb. 3).

Der Titel des Projekts: `Project title` wird zur Namensgebung für folgende Projekteigenschaften verwendet:

- **Das Unterverzeichnis:** Wird im Projektverzeichnis angelegt: `Resulting filename`
- **Die Projektdatei:** Enthält die Projekteinstellungen: `Project filename`

Compilerauswahl und Targetdefinition Nachdem der Projektname und Speicherort definiert wurden erscheint ein Dialog zur Auswahl des Compilers (vgl. Abb. 1.1.2). Dieser kann unverändert beibehalten und mit `Finish` die Erstellung des Projekts abgeschlossen werden.

1.1.3 Projekttypen

Die Wahl des Projekttyps legt die Art des Anwendungsprogramms fest, das erstellt werden soll. Bei einem nicht leeren

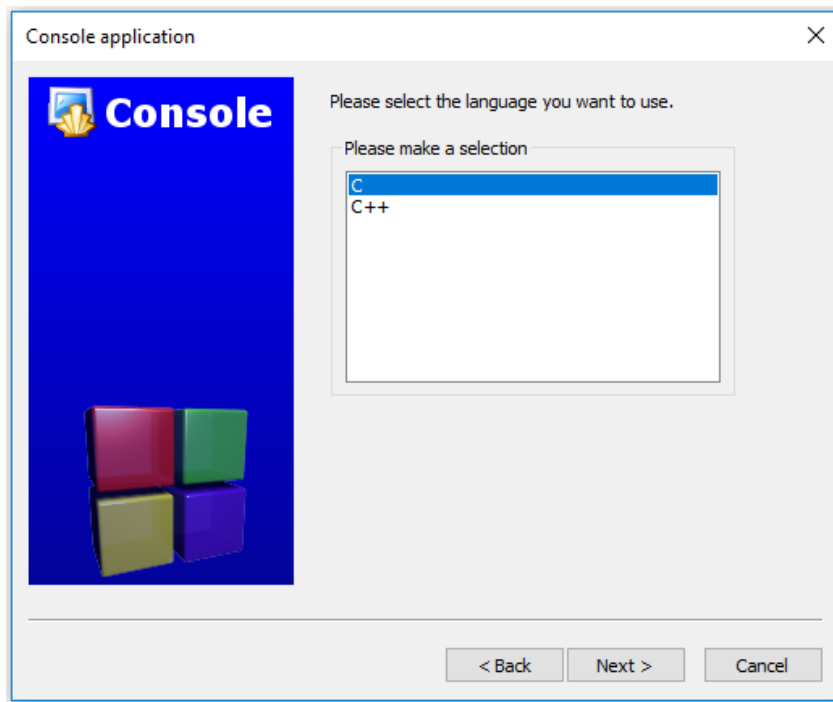


Abbildung 2: Dialog in CodeBlocks zur Wahl der Programmiersprache für ein neu erzeugtes Projekt. Achten Sie in diesem Semester darauf, hier immer C als Sprache zu wählen, sofern nicht anderes angegeben wird.

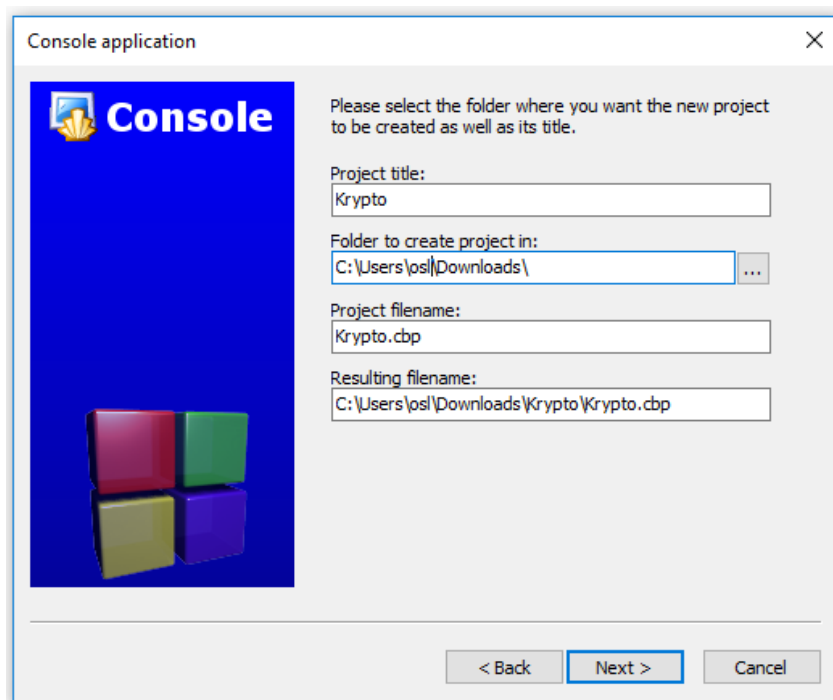


Abbildung 3: Dialog zur Konfiguration des Ortes im Dateisystem für das Projekt sowie des Projektnamens. Achten Sie unbedingt darauf, keine Leerzeichen oder Sonderzeichen zu verwenden.

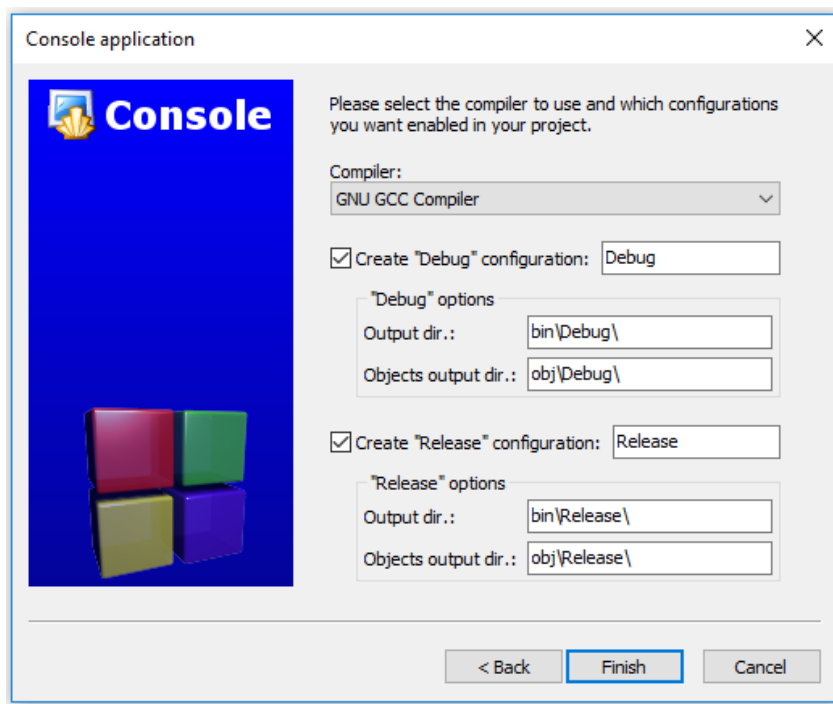


Abbildung 4: Auswahl der „Targets“ für den Übersetzungsvorgang. Hierbei handelt es sich um unterschiedliche Konfigurationen für den Übersetzungsvorgang. Hierbei können sehr komplexe Szenarien umgesetzt werden. Behalten Sie vorerst die Standardeinstellungen bei.

Projekt wird darüber hinaus auch ein Rahmen für ein Programm des betreffenden Typs erzeugt.

Console Application Dieser Projekttyp wird benutzt, um mit der Programmierung zu beginnen. Nach der Fertigstellung des Einrichtungsassistenten wird folgender Programmrahmen zur Verfügung gestellt:

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    printf("Hello world!\n");
    return 0;
}
```

Empty Project Dieser Projekttyp erzeugt ein leeres Projekt. Anschließend können Programmdateien in Code::Blocks neu erzeugt oder bereits vorhandene Dateien dem Projekt hinzugefügt werden (z.B. vorbereitete Programmdateien der Übungen).

- Neue C oder C++ Header-/ oder Quelldateien können über den Menüpunkt *File/New/File* erzeugt werden. Die neuen Dateien werden in den folgenden Dialogen zur „Release“- und „Debug“-Version hinzugefügt und der Pfad zum Speicherort wird angegeben
- Bereits vorhandene Programmdateien müssen in das Projektverzeichnis kopiert werden und anschließend durch *Project/Add Files* dem Projekt hinzugefügt werden.

1.1.4 Typischer Bearbeitungszyklus für Programme

Editieren In der Hauptarbeitsfläche von Code::Blocks werden die Programm-Quelldateien editiert.

Zusätzlich werden spezielle Unterstützungen für die Programmiersprache angeboten. Dazu gehören z.B. die Einfärbung von Schlüsselworten, Kommentaren und Zeichenkettenkonstanten (Syntax-Highlighting), aber auch eine Unterstützung zur Vervollständigung von Variablennamen bei der Eingabe.

Übersetzen Zur Erzeugung eines lauffähigen Programms müssen die Projektdateien übersetzt (kompiliert) werden.

Die Objektdateien des Compilerlaufes werden wie in der ersten Aufgabe (Aufgabe Null) gesehen in einem anschließenden Linker-Lauf zu einer ausführbaren Datei vereinigt („gelinkt“). Die aus Aufgabe Null bereits bekannten Objekt-Dateien finden sich im Verzeichnis `obj` und die ausführbare Datei im Verzeichnis `bin`.

In beiden Verzeichnissen gibt es für die Debug- und Release-Version jeweils einen Unterordner.

Zu jeder `.c`-Quelldatei finden wir im Erfolgsfall eine `.o`-Objektdatei mit gleichem Basisnamen. Die ausführbare Datei ist dabei das eigentliche Endprodukt des Übersetzungslaufs. Sie enthält die zusammen gebundenen `.o`-Dateien sowie die benötigten Unterprogramme aus den Bibliotheken.

Um Platz zu sparen, kann man für die Archivierung des Projektes die Object-Dateien, die ja lediglich Zwischenprodukte des Build-Vorgangs darstellen, mit *Build/ Clean* aufräumen.

Starten Wurde ein Projekt erfolgreich erzeugt, dann gibt es eine ausführbare Datei im bin-Verzeichnis. Das Programm kann gestartet werden, ohne die Entwicklungsumgebung zu verlassen. Dazu wird das Bedienelement für *Run* benutzt, wie in Abb. 5 gezeigt. Es öffnet sich ein Fenster, in dem die Ausgaben des Programms zu sehen sind.



Abbildung 5: Steuerleiste zur Übersetzung und Ausführung von Programmen. Das Zahnrad steht für die Übersetzung des Quellcodes in eine ausführbare Datei, der grüne Pfeil für das Ausführen der dabei erzeugten ausführbaren Datei.

Wichtig: Verwenden Sie nicht die Kombination aus *build&run*. Das führt leicht dazu, dass Sie die Warnungen des Compilers übersehen, welche aber ggf. Hinweise auf schwere Programmierfehler beinhalten.

2 Das erste Beispielprogramm

2.1 Verzeichnis für Übungsprogramme

Erstellen Sie, wie in Kapitel 1.1.1 beschrieben, ein Verzeichnis für Ihre Übungsaufgaben. Sie können das Verzeichnis aus Aufgabe 0 weiterverwenden.



Erstellen Sie ein Verzeichnis für ihre PIN1 Übungsprogramme. Sie können das Verzeichnis aus Aufgabe 0 weiterverwenden.

2.2 Ein erstes Beispielprojekt

Erstellen Sie, wie in Kapitel 1.1.2 beschrieben, ein Projekt mit folgenden Eigenschaften:

- **Projekttyp:** Console Application
- **Projektname:** Überlegen Sie sich einen sinnvollen Projektnamen für den Programmausschnitt
- **Speicherort:** Ihr zuvor erstelltes Verzeichnis für die Übungsaufgaben

Wichtig: Der Name des Projekts sollte die Aufgabe des Programms eindeutig identifizieren und kein Datum und keine Versionsnummer enthalten! Folgende Bezeichnungen für Verzeichnisse bzw. Programmnamen sind beispielsweise **nicht** sinnvoll:



Erstellen Sie ein neues Projekt und konfigurieren Sie es korrekt. Fügen Sie den angegebenen Beispielcode ein.

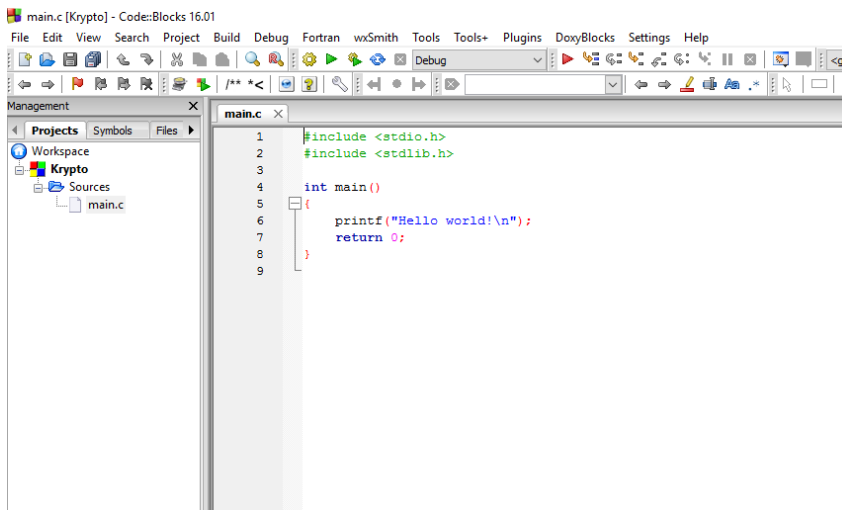


Abbildung 6: Die Entwicklungsumgebung Code::Blocks mit einem geöffneten und soeben neu erzeugten Projekt.

NeuesProjekt, NeuerOrdner,Krypto_neuerVersuch,
Summe_final_neu, Krypto_neu_fastfertig_versuch2,
Krypto_neu_fastfertig_versuch2_letzterVersuch.

Jetzt sollte Code::Blocks ähnlich aussehen, wie in Abbildung 6.

Überzeugen Sie sich davon, dass in Ihrem Projektverzeichnis ein Unterverzeichnis mit dem von Ihnen angegebenen Projektnamen angelegt worden ist und dass darin die beiden Dateien Ihr-Projektname.cbp mit den Projekt-Einstellungen und main.c als Quelldatei erzeugt wurden!

Fügen Sie nun folgendes Programm in die Datei main.c des eben erstellten Projekts ein.

```
#include <stdio.h>

/* Unser erstes Programm */
int main(void)
{
    int year = 2017;
    printf("Hello World in %d!\n", year);
    while (year <= 2019)
    {
        printf("Hello World in %d!\n", year);
        year = year+1;
    }
    return 0;
}
```

2.3 Kompilieren und Ausführen des Programms

Führen Sie den Programmcode wie in Kapitel 1.1.4 beschrieben aus.

Frage:

1. Was passiert, wenn man die Zeilen mit der Return-Anweisung und mit der `printf()`-Anweisung vertauscht?
2. Wie verhält sich der Compiler, wenn man eine geschweifte Klammer weglässt?
3. Was passiert, wenn man das Schließen des Kommentars weglässt?
4. Was passiert, wenn man einen Strichpunkt am Ende der Zeile vergisst?
5. Was passiert, wenn man das `#`-Zeichen in der ersten `include`-Anweisung weglässt?

Hinweis: Die Kenntnis des Verhaltens des Compilers in diesen Situationen liefert wertvolle Hinweise für die spätere Fehlersuche in eigenen Programmen.



Übersetzen Sie das Programm und führen Sie es aus. Erkunden Sie das Verhalten bei typischen Programmierfehlern.

3 Kennenlernen des Debuggers

3.1 Was ist Debuggen?

Die IEEE¹ definiert den Vorgang des Debuggen folgendermaßen:

debug: To detect, locate, and correct faults in a computer program. Techniques include use of break-points, desk checking, dumps, inspection, reversible execution, single-step operation, and traces.²

Die Fehlersuche in Programmen ist also so etwas wie Detektivarbeit und Softwareentwickler bedienen sich dabei der selben Methoden: Hinweise sammeln, Hypothesen aufstellen, Hypothesen prüfen, Abhilfen entwickeln.

Die in obiger Definition beschriebenen Maßnahmen zum Entdecken und Beseitigen der Fehler in Programmen lassen sich sehr grob in manuelle und durch Tools unterstützte Techniken aufteilen.

¹Institute of Electrical and Electronics Engineers, Berufsverband von Ingenieuren aus den Bereichen Elektrotechnik und Informationstechnik

²IEEE Standard Glossary of Software Engineering Terminology
<https://ieeexplore.ieee.org/document/159342>

Im weiteren Verlauf dieser Anleitung wird hier hauptsächlich auf die Verwendung eines Debuggers als Software-Werkzeug eingegangen.

3.2 Was ist ein Debugger?

Ein Debugger ist eine Software mit deren Hilfe andere Programme während ihres Programmablaufs kontrolliert und inspiziert werden können.

Dadurch wird es möglich, dem Programm bei der Arbeit „zuzusehen“ und so Ursachen für auftretendes Fehlverhalten des Programms aufzudecken.

Hinweis: Sie sollten nicht erwarten, dass der Debugger die Probleme selbständig findet oder löst, wie der Name vielleicht suggeriert. Es handelt sich dabei im Gegenteil lediglich um ein Werkzeug zur Informationsbeschaffung für die Fehlersuche.

Um mit Hilfe des Debuggers Fehler finden zu können, werden die Programme vom Debugger im Debug-Modus ausgeführt. Ohne das Ausnutzen weiterer Debugger-Funktionen ist dieser Modus kaum vom normalen Ausführen der Programme zu unterscheiden.

Als wichtigste Debugger-Funktionen sind das Setzen von Breakpoints (Haltepunkten) und das Betrachten von Variablenwerten („watches“) zu nennen. Mit Hilfe von Breakpoints, kann die Ausführung des zu untersuchenden Programms an den markierten Stellen unterbrochen werden.

Ein mit Hilfe eines Breakpoints unterbrochenes Programm lässt sich nach der Unterbrechung entweder normal weiter abarbeiten oder es kann in Einzelschritten durchgearbeitet werden. Bei der Abarbeitung des Programms in Einzelschritten stehen folgende Möglichkeiten zur Verfügung:

- **Einzelschritt (Step into):** Es werden alle Anweisung einzeln ausgeführt. Befindet sich ein Funktionsaufruf³ in der als nächstes auszuführenden Anweisung, so werden auch die Anweisungen in diesem Unterprogramm alle einzeln ausgeführt.
- **Prozedurschritt (Next line):** Es werden alle Anweisung einzeln ausgeführt. Befindet sich ein Funktionsaufruf in der als nächstes auszuführenden Anweisung, so wird dieses Unterprogramm komplett ausgeführt.

³Funktionen sind Programmteile, die separat und von verschiedenen Stellen des Gesamtprogramms aus ausgeführt werden können.

- **Ausführen bis Rücksprung (Step out):** Befindet sich die als nächstes auszuführende Anweisung innerhalb einer Funktion, wird diese Funktion bis zum Ende der Funktion in einem Schritt ausgeführt.

Der Nutzen der Einzelschritt-Ausführung eines Programms ergibt sich dadurch, dass während des Debug-Modus mit Hilfe von „Watches“ die Werte der Variablen⁴ des Programms während dessen Ausführung betrachtet werden können.

Als letzte Funktion ist hier noch der „Memory Dump“ zu nennen. Dieser ist in der Lage, ähnlich wie die Watches Speicherinhalte anzuzeigen. Bei einem Memory Dump wird ein Teil des Arbeitsspeichers direkt und ohne weitere Interpretation byteweise ausgegeben.

3.3 Debugger unter Code::Blocks

Breakpoints, Einzelschritt-Ausführung und Watches In Abbildung 7 ist die Symbolleiste des Debuggers zu sehen. Mit den Schaltflächen *Debug / Continue* (Roter Pfeil) starten Sie den Debug-Modus.

Anstelle der Buttons können Sie auch die Funktionstasten verwenden, die entsprechende Zuordnung der Tasten zu den Funktionen finden Sie im Menü unter „Debug“.

Hinweis: Mit den Funktionstasten bzw. Tastenkombinationen können Sie deutlich schneller arbeiten als mit der Maus.



Abbildung 7: Steuerleiste zum Debuggen von Programmen.

Die Schaltfläche mit dem roten Pfeil startet den Debug-Modus.

Bevor Sie den Debug-Modus starten, sollten Sie strategisch geschickt an den zu untersuchenden Stellen Breakpoints setzen.

Ein Breakpoint wird durch einen roten Kreis neben der entsprechenden Zeilennummer angezeigt. Den Breakpoint können Sie durch Anklicken des Bereiches rechts neben der Zeilennummer oder mit Hilfe der Funktionstaste F5 ein- und wieder ausschalten.



Testen Sie den Debugger und verfolgen Sie Ihr Programm bei der Arbeit.

⁴Variablen sind Behälter für Werte, wobei die in den Variablen gespeicherten Werte während des Programmablaufs geändert werden können um z.B. berechnete Ergebnisse abzuspeichern.

Ein Beispiel ist in Abb. 8 zu sehen.

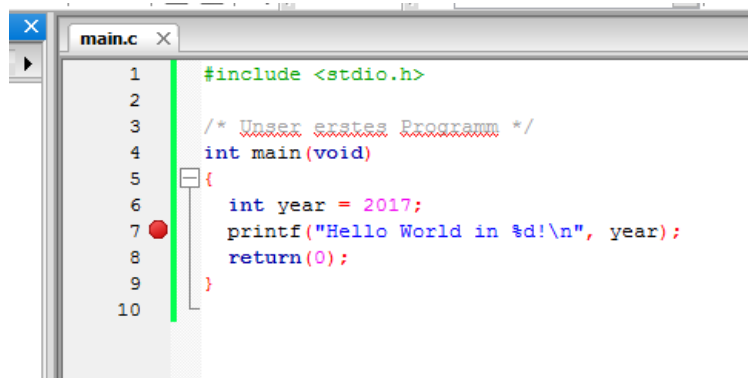


Abbildung 8: Dieser Bildausschnitt zeigt einen Breakpoint (roter Ball), eine Stelle an der ein im Debugger laufendes Programm angehalten wird, wenn die Ausführung die entsprechende Codestelle erreicht.

Nach dem Start im Debug-Modus wird das Programm bis zu einer mit einem Breakpoint markierten Zeile ausgeführt.

An dieser Stelle wird dann die Ausführung unterbrochen. Die unterbrochene Ausführung des Programms ist durch ein gelbes Dreieck, welches auf die nächste auszuführende Zeile mit Programmcode zeigt, zu erkennen.

Mit Hilfe der Buttons aus Abb. 7 kann die Ausführung des Programms fortgesetzt werden: Mit dem ersten Button *Debug/Continue* wird das Programm bis zum nächsten Breakpoint ausgeführt. Mit dem dritten Button *consoleNext line* wird das Programm bis zur nächsten Zeile Programmcode ausgeführt. Leere Zeilen und Zeilen mit geschweiften Klammern werden dabei ignoriert. Im Beispiel wäre die Zeile 8 die nächste Zeile.

Um nun während der Einzelschritt-Ausführung des Programms den Wert der Variablen betrachten zu können, müssen Sie im Menü unter *Debug/Debugging Windows* das *Watches*-Fenster aktivieren. In diesem Untermenü können außer den *Watches* auch weitere *Debugging*-Ansichten (Abbildung 9) aktiviert werden.

Um eine neue Variable hinzuzufügen, die beobachtet werden soll, klicken Sie in die erste freie Zeile des *Watches*-Fensters und geben dort den Namen der Variablen ein und bestätigen Sie die mit der Eingabetaste (Abbildung 10).



Beobachten Sie die Werte der Variablen während des Programmlaufs.

3.4 Untersuchen des Beispielprogramms

Setzen Sie in Ihrem zuvor erstellen Projekt einen Breakpoint bei der `int main(void)`-Zeile (vgl. Kapitel 3.3).



Untersuchen Sie Ihr Beispielprogramm mit dem Debugger.

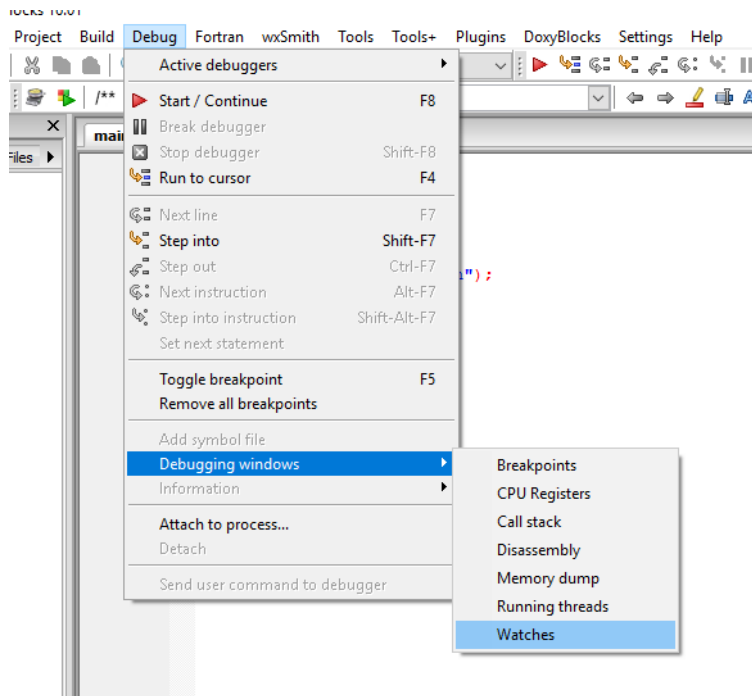


Abbildung 9: Aktivierung der verschiedenen Debugging Windows

Führen Sie nun das Beispielprogramm im Debug-Modus aus und aktivieren Sie das Watch-Window und machen Sie die Variable `year` sichtbar.

Analysieren Sie den Inhalt des Watch-Window und führen Sie das Programm zeilenweise bis zum Ende aus. Verfolgen Sie dabei die Änderungen im Watch-Window.

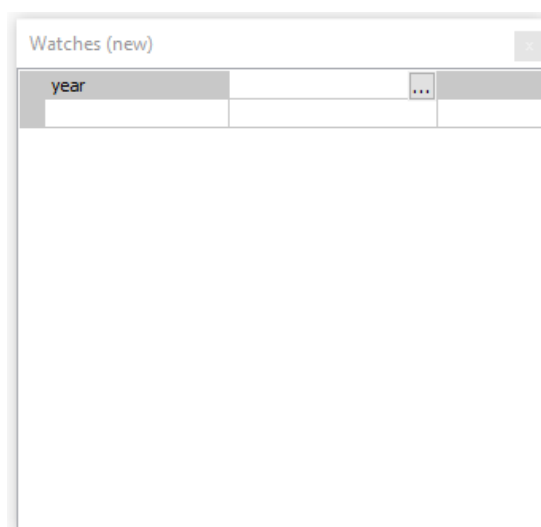


Abbildung 10: Watch Window zur Überwachung der Werte ausgewählter Variablen des Programms.